

DetecVFuzz: Enhancing Security in Consumer Electronic Devices Through Scalable Vulnerability Testing of Virtual Devices

Xi Xiao[✉], Yongjian Guo[✉], Alireza Jolfaei[✉], *Senior Member, IEEE*, Chuan Chen[✉],
 Mohammad Sayad Haghighi[✉], *Senior Member, IEEE*, Sheng Wen[✉], *Senior Member, IEEE*, and Yuanyi Lin

Abstract—The interconnection facilitated by consumer electronics has led to the creation of a complex network of devices. Testing efficiency of such consumer electronic devices can be improved through hardware emulation. QEMU is a classical emulator for simulation and testing. Fuzzing is a promising solution to test the security of such devices, which has become pivotal in identifying device vulnerability. However, Qtest lib-fuzzer, the QEMU native fuzzer, requires manual writing of stub code, which makes conducting the fuzzing process difficult and yields poor performance. To address these challenges in virtual device security analysis, we propose DetecVFuzz, a novel automatic fuzzing framework based on QEMU-Qtest, aiming to find vulnerabilities in virtual PCI devices, including USB, audio, network, etc. We inject mutated inputs into nested data structures as parameters of memory-related APIs to ensure the stable operation of VM procedures. We implemented DetecVFuzz and evaluated it on different virtual devices, such as virtual memory USB stack and drivers. In preliminary evaluations on nine recent versions of the QEMU, we found 15 new memory errors in virtual devices. In general, DetecVFuzz achieves 89.37% line coverage and 78.65% branch coverage in virtual devices. We identified 26 new crashes/errors, three of which were unseen new bugs.

Index Terms—Fuzzing, consumer electronics, computer security, application virtualization, QEMU testing, vulnerability.

I. INTRODUCTION

IN RECENT years, Internet-of-Things (IoT) devices have been widely adopted for supporting next-generation distributed systems due to the rapid development of low-cost consumer electronics and cloud computing [1]. This transformative shift extends across a spectrum of consumer electronics, ranging from smartphones to smart TVs and sophisticated home automation systems [2], creating online social networks and disseminating real-time information [3]. The interconnectivity facilitated by consumer electronics has given rise to an intricate Web of devices, necessitating a paradigm shift in computing infrastructure to meet the demands of this interconnected ecosystem. This is where device virtualization comes to the forefront, which can improve the testing efficiency of consumer electronic devices.

The device virtualization often relies on the QEMU emulator for simulation and testing during its development phase. As virtualization technology becomes ubiquitous, the spotlight on security challenges intensifies, particularly in the context of vulnerabilities inherent in virtualized devices. Notable instances, such as and the Venom vulnerability (CVE-2015-3456) [4], underscore the importance of addressing security concerns associated with virtualization platforms. These threats include cyber attacks against cyber-physical systems [5], [6], exhaustive privileges for USB devices (meaning that the devices might be granted excessive or unrestricted access or control within a system/virtualization platform), potential electrical attacks exploiting physical design flaws, and the exploitation of software vulnerabilities [7], [8], [9] within the host operating system [10], [11], [12].

Considering the paramount importance of virtual machine monitor (VMM) security, various fuzz testing tools have been proposed to detect bugs within VMMs [13]. VDF [14] is the first VMM fuzz testing framework, leveraging AFL to implement a coverage-guided virtual machine fuzzing methodology. Hyper-Cube [15] has designed a multi-dimensional, platform-agnostic fuzz testing approach based on a custom operating system. Although Hyper-Cube does not apply coverage-guided fuzz testing techniques to its fuzzing process, its performance still surpasses VDF due to its high-throughput

Received 14 March 2024; revised 7 July 2024 and 25 October 2024; accepted 24 November 2024. Date of publication 3 January 2025; date of current version 7 November 2025. This work was supported in part by the Key Laboratory of Computing Power Network and Information Security, Ministry of Education under Grant 2023ZD034, and in part by the Major Key Project of PCL under Grant PCL2023A06-4. (*Corresponding author: Xi Xiao.*)

Xi Xiao and Yongjian Guo are with the Shenzhen International Graduate School, Tsinghua University, Shenzhen 518055, China, also with the Key Laboratory of Computing Power Network and Information Security, Ministry of Education, Qilu University of Technology, Shandong Academy of Sciences, Jinan 250316, China, and also with the Department of Advanced Interdisciplinary Research, Peng Cheng Laboratory, Shenzhen 518066, China (e-mail: xiaox@sz.tsinghua.edu.cn; guo-yj24@mails.tsinghua.edu.cn).

Alireza Jolfaei is with the College of Science and Engineering, Flinders University, Adelaide, SA 5042, Australia (e-mail: alireza.jolfaei@flinders.edu.au).

Chuan Chen is with the Key Laboratory of Computing Power Network and Information Security, Ministry of Education, Shandong Computer Science Center, National Supercomputer Center in Jinan, Qilu University of Technology, Shandong Academy of Sciences, Jinan 250316, China (e-mail: chenchuan.3000@163.com).

Mohammad Sayad Haghighi and Sheng Wen are with the School of Science, Computing and Engineering Technologies, Swinburne University of Technology, Melbourne, VIC 3122, Australia (e-mail: msayadhaghighi@swin.edu.au; swen@swin.edu.au).

Yuanyi Lin is with the Shenzhen International Graduate School, Tsinghua University, Shenzhen 518055, China (e-mail: yy-lin20@mails.tsinghua.edu.cn).

Digital Object Identifier 10.1109/TCE.2025.3525512

design. However, the core philosophy of both Hyper-Cube and VDF is consistent: they generate a set of random values at the basic interfaces (MMIO, DMA, etc.). In reality, they are unaware of the protocol implementation of virtual devices and are uncertain about how to generate data structures transmitted via DMA, hence such automated testing often generates a multitude of invalid samples, making it difficult to trigger real vulnerabilities.

Nyx [16] understands the protocol of the target device and constructs structured fuzz tests based on the specifications provided by the user. However, it requires extensive manual work to create the specification templates. For instance, the authors of Nyx mentioned that they spent approximately two days on the most complex specifications. Consequently, Nyx cannot be extended across different devices, as it necessitates manual rule-writing for each new protocol. This is a common drawback of structured fuzz testing, whose effectiveness heavily relies on the completeness of the nested structures, which are typically manually written based on the developers' understanding of the protocol specifications. Generally, developers need to extract all types of basic data structures from the device protocol, including the connection relationships between basic structures and the pointer offsets within each data structure. Applying structured fuzz testing to VMMs, such a labor-intensive process is both time-consuming and prone to errors. Therefore, existing fuzz testing methods are unable to effectively test virtual devices.

To address these challenges, we introduce the DetecVFuzz fuzzing framework. DetecVFuzz seamlessly integrates with the hypervisor, capturing memory-related APIs and injecting mutated test cases into complex nested data structures as inputs for these APIs. This orchestration enables continuous interaction between the hypervisor and virtual devices using novel inputs. Notably, our approach achieves an impressive average line coverage of 89.37% and a branch coverage of 78.65%, contributing to the robust security posture of virtualized systems in consumer electronics. In summary, this work makes the contributions:

- We propose a scalable virtual machine monitor fuzz testing framework that can automatically decompose nested structures and create multiple seed pools based on the type of input structure to ensure compliance with various input parameter types, solving the problem of manually writing nested structures based on the understanding of protocol specifications in structural fuzz testing.
- Furthermore, we introduce DetecVFuzz, which allows fuzz testing to be initiated externally for the virtual machine/device. Subsequently, DetecVFuzz interacts with virtual devices using a fuzz testing proxy and bypasses MMIO/PIO initialization to achieve a rapid and efficient fuzz testing loop, enabling high-speed fuzz testing. This provides an idea for the subsequent vulnerability testing in simulated consumer electronic devices.
- Compared to the VDF and hypercube, DetecVFuzz achieves higher code coverage across multiple QEMU virtual devices. Additionally, DetecVFuzz has identified 15 new memory errors in the virtual devices that had previously undergone extensive fuzz testing.

The rest of this paper is organized as follows. Section II reviews the related work. Section III provides the background of PCI hardware analysis and our insights into DetecVFuzz with a running example. Section IV presents the detailed design of DetecVFuzz. The evaluation results are summarized in Section V. Section VI discusses some limitations of the current design and the possible improvements, and finally, Section VII concludes this paper.

II. RELATED WORK

Fuzzing is currently the most popular and effective method for discovering vulnerabilities, as it involves generating random inputs as seeds to test programs [17]. This approach is known for its high efficiency in detecting bugs and vulnerabilities in modern software [18], [19], [20], [21]. Researchers have explored various aspects, including selecting appropriate fuzzing seeds [22] and employing concolic fuzzing to identify software vulnerabilities [23], all of which have contributed to improving coverage and bug discovery.

Device fuzzing can drive vulnerabilities in programs or firmware, thereby facilitating interaction with the operating system. Agamotto [24] utilizes virtual machine images to restore checkpoints, enabling rapid fork operations for kernel driver fuzzing. PeriScope [25] injects random data at the DMA/MMIO interfaces, but it is closely tied to specific operating systems and kernel versions. However, it lacks the capacity for intelligent fuzzing and does not collect coverage feedback. USBFuzz [26] manages to fuzz USB firmware on the guest operating system kernel, similar to the AFL QEMU mode [27]. LibFuzzer [28], a fundamental fuzzer, employs AddressSanitizer [29] to effectively detect memory errors. Subsequently, various kernel fuzzers, such as syzkaller [30], TriforceAFL [31], trinity [32], DIFUZE [33], kAFL [34], and RAZZER [35], have been developed to fuzz system call arguments and have successfully uncovered numerous bugs in popular operating system kernels. Moreover, researchers have extended fuzzing to a wider range of targets, spanning from kernel-level applications [34], [36], [37], [38], [39] to IoT [40], [41], [42].

However, fuzzing faces a significant challenge when mining for vulnerabilities in virtual machine monitors, namely, the difficulty of providing random input from devices, which limits the effectiveness of fuzzing on the devices within. Dedicated programmable hardware devices, such as FaceDancer [43], are effective but expensive and lack scalability, as each device can only fuzz a single target. Some solutions have adapted the kernel, for example, syzkaller [30], which includes a USB fuzzer [26] that injects random data into the USB stack via extended system calls. VDF [14] integrates AFL into VM fuzzing and records VM address initialization to expedite tests. Hyper-Cube [15] introduces a custom operating system for multi-entrance fuzzing, which, despite its high throughput, faces challenges when exploring complex devices due to its black-box design. The presence of nested structures in virtual machines makes it difficult for automatically generated inputs to test the correct code paths, Nyx [16] and Nyx-net [44] focus on input structure, striving to replicate the execution

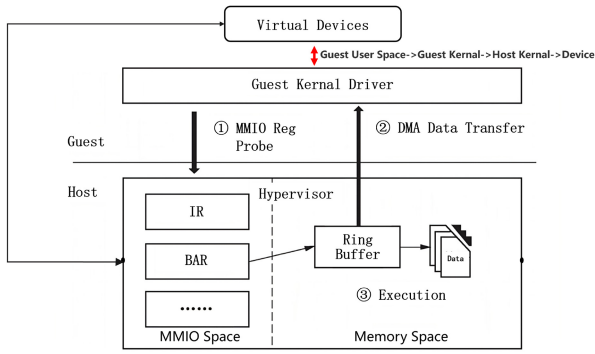


Fig. 1. How virtual devices, hypervisor and drivers communicate during initialization and data transferring.

procedures to enhance the efficiency of discovering undefined behaviors. Nevertheless, these methods involve more complex setup processes and require significant manual work. Our proposed DetecVFuzz can automatically complete the testing of nested data structures without manual settings, and the test results are not inferior to these methods, and it is scalable.

III. BACKGROUND AND MOTIVATION

While the virtual machine technique is widely adapted to different architectures, such as QEMU-KVM used in OpenStack for the compute nodes [45], the potential vulnerabilities might arouse severe consequences.

Virtual devices in modern hypervisors emulate hardware peripherals, providing additional functionality for guest VMs. They act like real hardware, allowing guest OS drivers to interact with them as if they are physical devices. Virtual devices communicate with the OS via unique register interfaces defined by their protocol specifications. Since virtual device emulation occurs at the host level and guests can access them with arbitrary data, they represent the largest attack surface for hypervisors.

As an example, CVE-2020-14364 is an out-of-bounds read/write access flaw that was found in the USB emulator of QEMU in versions before 5.2.0. This issue occurs while processing USB packets from a guest when USB Device *setup_len* exceeds its *data_buf[4096]* in the *do_token_in*, *do_token_out* routines. This flaw allows a guest user to escape from the virtual machine.

The difficulty of fuzzing hypervisors and their virtual devices lies in the diverse virtualization environments where the fuzzing details need to be adjusted for different devices. By clarifying the difficulties in fuzzing virtual devices, we designed a framework for fuzzing specific devices.

At present, fuzz testing usually generates random unstructured data. Even the data generated after guidance based on coverage is highly unstructured. This is sufficient and effective for conventional programs, but for interactive programs, some standards, and data that do not follow the norms will be discarded. Such a fuzzer is inefficient. The virtualized devices simulated by QEMU all have their own protocols. Fig. 1 shows how virtual devices interact and transfer data with QEMU. With DMA (Direct Memory Access), drivers can bypass the CPU and accomplish complex data delivery within the related

TABLE I
THE NUMBER OF VIRTUAL DEVICE TYPES THAT
SUPPORT DMA OPERATIONS

Type	Device	Count
USB	UHCI, EHCI, OHCI, XHCI	4
Storage	Esp, AHCI, lsi53c810 Megasas, mptsas, NVME, virtio	12
Network	e1000, e1000e, eepro100 PCNET, rocker, rtl8139, ne2000	11
Audio	ac97, cs4231a, es1370, intel-hda, sb16	2
Total		29

communication protocols. If unprocessed random data is used, most of the input must be invalid, which wastes resources and time.

However, the function call logic of the analog device is complex, especially for complex protocols, often requiring specific inputs to reach certain processes of some functions. Even structured input data may take a long time to mutate into specific code blocks. Therefore, a method is needed to guide the fuzzer to the specified code block.

In general, a virtual device exposes three important interaction interfaces to the client: memory-mapped I/O (MMIO), ported I/O (PIO), and direct memory access (DMA). Fig 1 shows the general workflow of virtual machine transactions. When the device starts to execute, the client driver usually writes data to the MMIO or PIO area to let the device perform some initialization work, such as device status setting and address register initialization. The target is typically the pre-allocated buffer in the client. Once the initialization phase is complete, the device enters a state where it is ready to process data. During this phase, the device starts to perform device-specific work, such as transferring USB data and sending network packets. The primary interaction mechanism during this data processing phase is DMA, which allows the device to transfer large amounts of complex data to the client. Since the data processing part is the main function of the device, it contains most of the code paths. Therefore, this part is more likely to introduce security risks than other parts.

To demonstrate that the hypervisor's virtual devices make extensive use of DMA, we performed a statistical analysis of the percentage of devices in QEMU that support DMA communication. We selected the five most popular QEMU device classes used in virtualization scenarios, excluding some miscellaneous devices and backend devices. We manually analyzed whether there is a DMA transmission mechanism in the device. Among the 39 commonly used USB, storage, network, and audio devices, 74.4% of the virtual device codes support DMA. The specific results are shown in Table I. Except for display devices, almost all devices must use DMA to transfer complex data structures, especially those related to storage and network operations. Therefore, DMA is widely used in hypervisors, which necessitates paying more attention to DMA-related code when fuzzing hypervisors.

IV. FUZZING APPROACH

A. System Overview

The entire fuzzing framework is illustrated in Fig. 2. First, DetecVFuzz integrates with the hypervisor's APIs related to

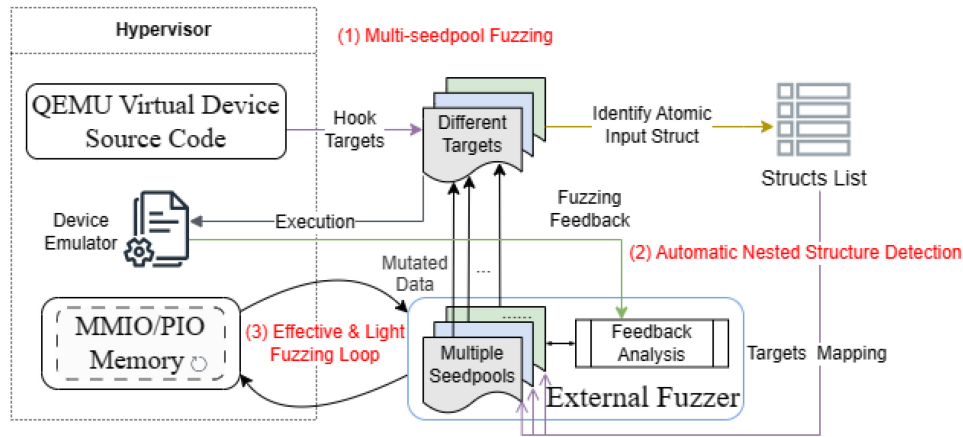


Fig. 2. The flow of fuzz loop in DetecVFuzz.

device I/O from the source code, as these APIs are responsible for extracting data from the MMIO buffer and executing device functions. These selected target APIs serve as entry points for DetecVFuzz. In virtual devices, these I/O APIs consist of DMA and PCI read/write functions, acting as the bridge between the client operating system's memory and the virtual devices. They are replaced through our hooking operations.

Subsequently, DetecVFuzz constructs seed pools for atomic data structures that appear in the device. By analyzing the context of the targets, we detect all single and independent minimal input structures from the raw inputs. The fuzzer maps the target functions to their corresponding input structures, which correspond to specific seed pools.

Finally, fuzzing begins with an external AFL fuzzer, which selects seed pools for each target and generates test cases. The device emulator processes requests and executes transactions. The program generates feedback from the fuzzing, which improves the seed pool structure and initiates the next iteration of fuzz testing. In the following section, we elaborate on three core procedures in our fuzzing. Corresponding to steps (1) (2), and (3) in Fig. 2.

B. Multi-Seed Pools Fuzzing

In a virtual device, a specific DMA operation retrieves a large number of inputs from the guest space. These inputs often contain numerous nested structures, where field members in one structure point to another structure. The extensive use of nested structures makes traditional fuzzing ineffective because it requires a precise memory layout of complex nested structures. This approach is not friendly to traditional fuzzing and demands significant manual effort.

More specifically, when the fuzzer generates random data structures, including random pointer fields that indicate the following structure, the execution often halts due to invalid memory access. Providing syntactically valid structure information for mutators is a labor-intensive task and is not scalable because different virtual devices have varying data structure specifications.

To address this issue, we intercept the device's access to the guest's memory by hooking the related DMA API

functions, which we refer to as our "TARGETS." Operating based on the source code of the hypervisor, DetecVFuzz integrates with the hypervisor's DMA mechanism and converts DMA transfers into reads from inputs generated by our fuzzer. Specifically, we select the DMA-related APIs (e.g., `pci_dma_read` and its wrapped functions) as patterns and insert macros into the source code of the target device. Then, all DMA-related APIs are replaced by macros with methods that read mutated inputs from external files, produced by our external fuzzer. Each DMA request-target is responded to with a mutated input generated by the traditional coverage-guided fuzzer, regardless of where the pointer points.

Functions related to DMA operations are responsible for transferring data between the guest and host machines and are a key mechanism for constructing nested data structures. By capturing these target functions, we gain complete control over the DMA operation interface, and any DMA request will be responded to with test cases generated by the coverage-guided fuzzer, bypassing the DMA addressing operations. When facing a DMA read request at runtime, the following procedure is executed:

(1) DetecVFuzz ensures that the DMA read function call originates from the device under monitoring, and it disregards DMA transfer requests from other internal system components.

(2) Given the host process buffer, the buffer's size, and the specific DMA API, DetecVFuzz retrieves the appropriate data from a specific seed pool. We will explain how to determine the mapping relations between targets and seed pools in the next section.

Fig. 4 illustrates the process of injecting mutated data for each target API. For different input structures, we have obtained the specific node structure types of DMA operation parameters during execution through static analysis. Based on replacing the inputs of the corresponding target functions with test cases, DetecVfuzz also replaces DMA transfers with inputs generated by the fuzzer, while introducing parameter type constraints. For example, `read_from_testcase(&buf, size)` is a typical DMA hook code with type constraints, which replaces the original `dev`, `buffer_addr`, and `&buf` as inputs for the PCI read operation.

This process disassembles the complex and extensive inputs, making it easier for the fuzzer to adapt to the function inputs and explore deeper paths with fewer loops. In this way, DetecVFuzz provides a flattened sequence of DMA data for hypervisor fuzzing. It transforms all nested structures into sequences of single-unit structures and enhances the target API with a multi-seed pool fuzzer while preserving the semantic correctness of nested structures.

In each fuzzing iteration, the fuzzer engine initially generates sequences of mutated DMA data. Subsequently, the device begins traversing the nested structure. Each target function within the structure is relinked to the fuzzer, taking a mutated fuzzing test case from its corresponding seed pool. Consequently, all code paths that include the target DMA requests can be effectively covered.

DetecVFuzz treats all data structures as a collection of metadata without considering the pointers, similar to userspace programs. The content of each node in a higher-level tree structure is generated randomly, including the pointer field. As shown in Fig. 4, the pointer to a buffer and the pointer link between nodes are both randomly generated. Each time a pointer is addressed, it will be re-linked to the mutated inputs. This design makes the fuzzing process compatible with the deep path requiring nested structures.

C. Automatic Nested Structure Detection

Using the DMA hook to assign mutated data to different targets, we have successfully created a fuzzing process that can effectively avoid nested data. However, we did not take the parameter types into account when organizing DMA inputs, which introduces an efficiency problem. As described in Section III, the node types in nested structures are dynamically determined, meaning that the program's control flow changes dynamically based on previously consumed DMA test cases.

Suppose the hypervisor requests structure A first and then structure B in the current iteration. This leads coverage-guided fuzzers to teach themselves to generate seeds with semantics close to the combination sequence of A and B. However, if the hypervisor requests structure A first and then structure C in the next iteration, the execution flow will pass the path with A but fail in the path with C. Consequently, the fuzzer-generated structure B would be rejected by the hypervisor, as structure B presents semantically invalid data for the requested structure C. To address this issue, we propose a structure detection fuzzing method. Essentially, DetecVFuzz aims to provide type awareness for the multi-seed pool fuzzer so that it can dynamically generate the correct test case according to the requested data type of the program.

To achieve this, DetecVFuzz first decouples the nested structures into independent atomic structures by using a further DMA hooking method. Next, it initializes different seed pools, one for each type of atomic structure. Then, with the type guidance, DetecVFuzz performs automatic fuzzing on the hypervisor. This idea is based on the rule of devices and DMA operations: each atomic function structure is independent, and there is no dependency between them. Thus, this decoupling method does not compromise the semantics

of the entire nested structure. Specifically, automatic nested structure detection consists of three main steps: Identifying atomic structures, Type Constraints, and Target & Structure Mapping.

1) Identify atomic structure. To collect knowledge of structure types, we retrieve type information indicated by each DMA operation from the source code.

In an effort to identify all the DMA atomic structures, DetecVFuzz conducts a static analysis on the hypervisor's source code. This analysis specifically involves the adaptation of a unique data flow analysis approach, which is essentially a form of live-variable analysis. We initiate this analysis by taking the host's buffer field of DMA operations (e.g., `pci_dma_read`, and its wrapped function) as the source and perform a backward data flow analysis from this source to its declaration.

Once all atomic structures are gathered, we assign unique IDs to each of them. These assigned IDs are instrumental in identifying the structure type of each DMA request at runtime, ensuring that each target will be associated with the correct seed pool.

2) Type Constraint. With the assigned unique IDs, DetecVFuzz can determine the specific type of structure required during DMA transfers within the fuzzing loop. Building upon the previous DMA hooking, DetecVFuzz introduces additional type constraints when translating DMA transfers into reads from mutated input.

By implementing these additional type constraints, DetecVFuzz ensures that each memory read operation is limited to retrieving data from the corresponding seed pool, based on the node type, rather than following a predefined DMA sequence. This approach effectively breaks down the nested structures into individual nodes and organizes them into categories according to their node type.

3) Targets & Struct Mapping. To accommodate multi-object inputs (comprising more than a single file input), we made modifications to the basic AFL to enable support for multi-seed pools in parallel. This enhancement allows the fuzzer to independently mutate each seed pool for various types of program inputs, guided by coverage feedback. Consequently, the fuzzer can swiftly adapt its inputs to mimic specific data structures within each object in different seed pools. Even when the program dynamically requires different types of data as input during execution, the fuzzer can consistently provide semantically valid inputs from the corresponding seed pool.

This seed pool-based method builds upon the existing coverage-guided fuzzing technique used in AFL and introduces parallelism, enabling it to handle multiple targets concurrently. Each seed pool operates independently and adheres to the same mutation strategy.

Example: In Fig. 3, we present the structure detection fuzzing in USB-UHCI in detail. As a first step, we list three DMA objects (`qh`, `td`, and `last_td`) which we found by live-variable analysis. Specifically, since `qh` holds the user-supplied buffer through `pci_dma_read` in `uhci_process_frame`, we replace the correspondent DMA read function that serves for the object `qh` with `pci_dma_read(&qh, sizeof(qh), 1)`.

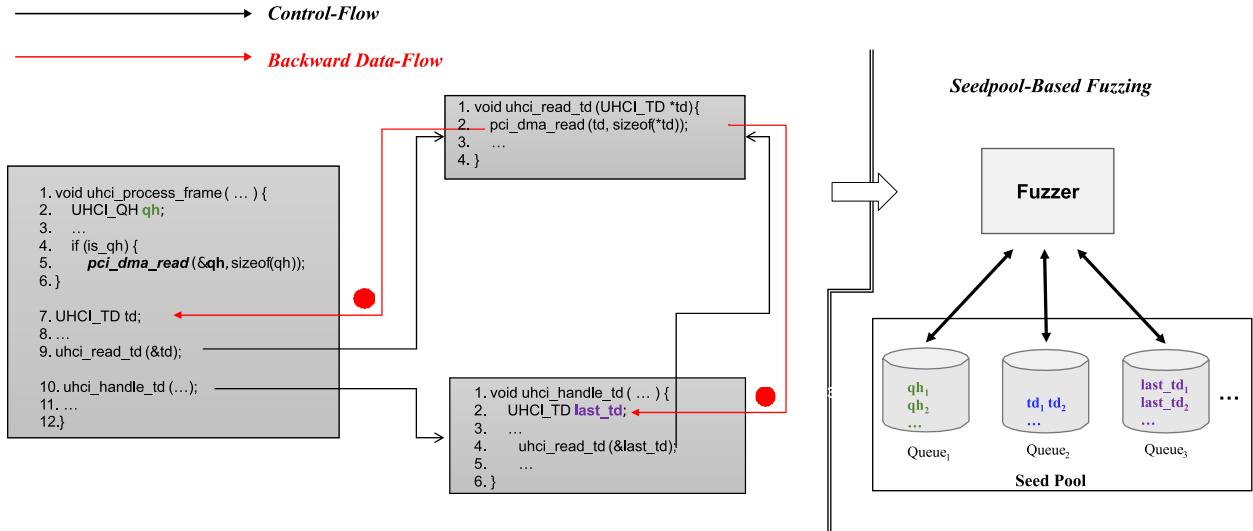


Fig. 3. An example presenting the semantics-aware fuzzing via seed pools.

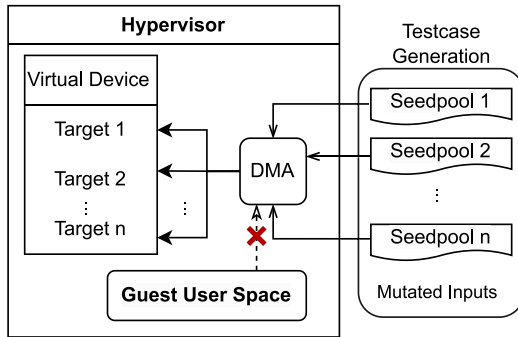


Fig. 4. Splitting nested inputs into different seed pools targeting multi-DMA functions.

In addition, since `td` and `last_td` hold the user-supplied buffer through `dma read` in `uhci_read_td` (`pci_dma_read` here serves for multiple objects in a wrapped function), we replace the function call `uhci_read_td(&td)` that serves for the object `td` with `uhci_read_td(&td,2)`, replace the function call `uhci_read_td(&td)` that serves for the target object `last_td` with `uhci_read_td(&td,3)`, and then replace `pci_dma_read(td, sizeof(*td))` with `pci_dma_read(td, sizeof(*td),id)`. In this way, these three kinds of target functions represent nodes with different structures in the nested memories buffer.

D. Effective & Light Fuzzing Loop

So far, we have built a complete `multi_seed` pool fuzzer and matched each target to a single seed pool that matches its input structure. However, according to the content mentioned in Section III, different from ordinary software programs, fuzz testing on a virtual machine still has performance problems in data transmission and process startup due to the characteristics of the virtual machine itself. There are some limitations when running the fuzzer agent on the guest OS:

This approach degrades performance due to the frequent use of VM-exits. When the guest VM needs to access the hardware, it triggers a trap that causes a VM-exit in the host

kernel. The VM-exit transfers control back to the hypervisor, which emulates the privileged operation on behalf of the VM. Consequently, every access request in the guest system results in a “heavy-weight exit” to the hypervisor. (1) This approach increases the complexity of implementation and can lead to communication instability, as it requires establishing a communication channel between the host and guest for transmitting fuzzed instructions. (2) DetecVFuzz, however, aims to optimize the fuzzing loop for maximum efficiency. In contrast to conventional fuzzing methods that often run in the same domain as the target program, we modified the hypervisor. DetecVFuzz places the fuzzing agent on the external host, rather than inside the virtual machine.

DetecVFuzz hooks into the hypervisor’s API used to initialize MMIO/PIO regions. When the device initializes as the virtual machine starts, DetecVFuzz retrieves callbacks for MMIO/PIO operations (read/write). We consider these as entry points for fuzzing because MMIO and PIO are the primary entry points for drivers to interact with the hardware. DetecVFuzz utilizes these entry points to drive the fuzzing loop, where the fuzzer interacts with the device.

This method accelerates virtual machine fuzz testing and eliminates the time-consuming virtual device restart and initialization process. Fig. 5 illustrates the principle schematic diagram of skipping the MMIO/PIO initialization process in the scheme. We record the device initialization information stored in MMIO and PIO and note the corresponding address and offset of the device. In each subsequent fuzzing cycle, we skip the original initialization step and directly copy the initial result to the corresponding address, allowing for a quick load of the virtual machine environment.

We save the state information in MMIO/PIO during the initialization of the virtual machine and virtual device to enable the quick restoration of the running state. This approach allows the external fuzzer to inject mutated data without interference, facilitating the repeated execution of the fuzzing loop. As a result, DetecVFuzz can be tested persistently and efficiently.

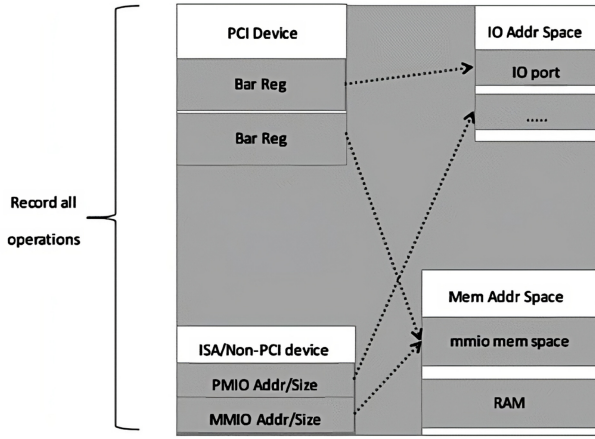


Fig. 5. Record and bypass setup and initialization operations.

Based on the above analysis, DetecVFuzz integrates structure detection and a multi-seed pool-based fuzzing engine to conduct structure detection fuzzing on hypervisors. The fuzzing process follows a typical client-host model, where DetecVFuzz (the host) manages incoming DMA requests from the target hypervisor (the client). The main steps of the fuzzing cycle are outlined in Fig. 6, encompassing the following key stages:

(1) DetecVFuzz establishes all essential seed queues and initializes the global coverage map during the initialization phase (Initialize and Entry Setup). (2) DetecVFuzz waits for DMA requests from the target hypervisor and prepares for the mutation input in subsequent processes. (3) DetecVFuzz selects seeds from the respective seed queue and mutates them to generate new candidate seeds. (4) DetecVFuzz supplies these new candidate seeds to the target program, monitoring coverage information. A candidate seed that explores new coverage is deemed valid and placed in the relevant seed queue. This approach enables each basic seed queue to learn from scratch and gradually produce valid seeds that match its specific type.

Once this algorithm converges, we typically acquire semantically valid inputs for each type of DMA object, enhancing the overall efficiency of fuzzing. In the seed scheduling phase, DetecVFuzz automatically restores connections between seeds from different seed pools by maintaining references between the currently visited torrent and the previously visited one, ensuring reliable and reproducible crashes. The above steps are repeated (loop request) to continuously search for potential errors in virtual device programs.

V. EVALUATION

We perform experiments on hardware simulated by QEMU to answer the following research questions:

- 1) Is DetecVFuzz scalable when fuzzing hypervisor virtual devices, and what is the vulnerability-finding capability of DetecVFuzz?
- 2) How much do the two main modules in DetecVFuzz, the multi_seed pool fuzzing framework and the fuzzing

module for automated structure detection, contribute to the coverage results of fuzzing, and what are their respective performances?

- 3) What is the performance gain of DetecVFuzz compared to the state-of-the-art hypervisor fuzzing tools such as Nyx and VDF?

We implemented our design and strategies through a prototype created in C/C++ code, adhering to the general structure of AFL [34] on version 2.51b for the fuzzing process. To facilitate coverage-guided fuzzing, we conducted instrumentation of the hypervisor's code to collect feedback information using AFL and its accompanying compilation tools. When the virtual machine initializes, the instrumented code in the target writes the coverage feedback to a bitmap, which is exported as a shared memory area accessible by the fuzzer component. It's worth noting that we only instrumented and recompiled the programs of virtual devices, considering performance optimization.

To enhance the initialization efficiency when the fuzzer begins, we collected initial seeds of valid test cases under standard full-system emulation. This involved logging all accesses to the target device through DMA and MMIO/PIO. It's important to mention that this step is optional, as one can start from any arbitrary seed or craft test cases independently.

Our experiments were conducted on a machine equipped with a 2.20 GHz, 18-core Xeon processor, and 128 GB of RAM running Ubuntu 18.04 LTS. We targeted certain virtualized devices within the QEMU 5.0.0 simulation (as shown in Table II), and built them using AddressSanitizer [29] to detect memory errors. Each individual target was executed for 24 hours and repeated ten times. The reported performance metrics are based on the average results [46].

A. Scalability

We carry out large-scale experiments to demonstrate the scalability of DetecVFuzz (Q1). We apply DetecVFuzz to a dozen QEMU virtual devices. The code coverage and performance overhead statistics data in Table II shows that DetecVFuzz can be easily configured for various virtual devices fuzzing setup and efficiently promote the fuzzing process.

To examine the ability of DetecVFuzz, we perform experiments in QEMU to discover code coverage in 24h fuzzing. We use gcov to measure branch coverage [47].

We selected popular QEMU devices for evaluation. The selection of these devices was based on the following characteristics: popularity within the community, the level of development activity, and the diversity of categories. These devices are representative, including audio, graphics, network, USB, and storage devices, and they cover standard virtualization scenarios such as cloud computing, virtual private server hosting, and desktop virtualization. For instance:

- Floppy: The floppy drive emulation in QEMU is a device that, although rarely used in modern computers, is crucial for meeting compatibility needs in specific virtualization scenarios.

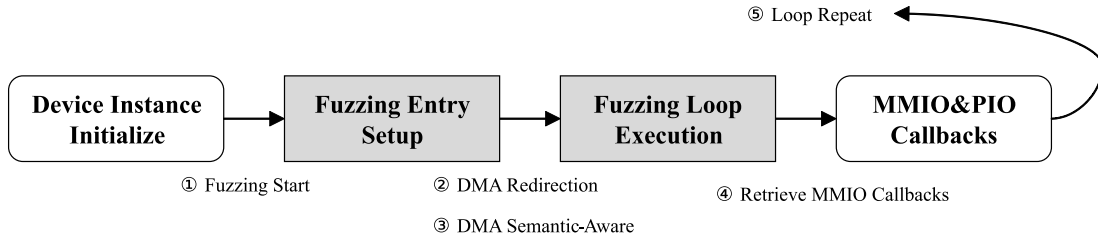


Fig. 6. DetecVFuzz fuzzing loop.

TABLE II
THREE CODE COVERAGE INDICATORS (LINE, FUNCTION AND BRANCH) OF DETECVFUZZ AND THE
EXECUTION SPEED ON TARGETING VIRTUAL DEVICES

Type	Device	Line Coverage	Function Coverage	Branch Coverage	Speed (times per second)
Block	Floppy	97.24%	98.30%	79.17%	6257.38
Audio	SoundBlaster	98.90%	100.00%	78.44%	3658.44
	intel-hda	96.69%	95.25%	80.12%	8239.60
	ac97	94.51%	92.17%	91.80%	2945.35
	sb16	83.29%	94.60%	84.46%	6351.23
	cs4231a	96.25%	99.50%	95.75%	3743.38
	es1370	95.17%	93.22%	92.06%	3862.18
USB	UHCI	88.98%	87.65%	83.62%	9596.96
	OHCI	85.33%	92.45%	81.32%	5382.97
	EHCI	86.62%	91.54%	73.42%	3859.58
	XHCI	89.40%	96.16%	89.20%	7372.92
Storage	NVME	64.74%	79.69%	62.28%	8019.26
	AHCI	88.69%	83.65%	63.40%	5533.59
	SDHCI	88.45%	94.60%	89.07%	2386.16
	Megass	64.50%	78.65%	54.38%	4237.50
Network	PCNET	97.85%	100.00%	89.22%	4764.54
	e1000	95.40%	97.80%	78.37%	1083.52
	RTL8139	92.35%	95.23%	84.66%	5690.51
	ne2000	97.57%	98.55%	94.12%	5264.29
	eepro100	96.52%	88.46%	78.36%	2382.08
Display	ati-vga-2d	51.76%	39.16%	25.44%	2034.95
	virtio-GPU	54.37%	57.26%	50.82%	1533.84
VirtIO	virtio-blk	62.28%	65.42%	54.49%	1204.52

- SoundBlaster: The emulation of the Creative Technology Sound Blaster series sound card, which provides audio output functionality and is widely used in PC audio hardware.
- ac97: Integrated sound card emulation that uses the Audio Codec '97 standard to provide high-quality audio playback and recording capabilities.
- UHCI: Universal Host Controller Interface emulation, simulating a USB 1.1 controller, allowing virtual machines to communicate with external devices through the USB interface.
- NVME: Emulation of the Non-Volatile Memory Express interface, a high-speed storage device interface commonly used for solid-state drives.
- PCNET: Network adapter emulation, simulating the AMD PCnet series Ethernet controller, providing network connectivity for virtual machines.
- ati-vga-2d: Graphics adapter emulation, simulating the ATI Technologies VGA-compatible 2D graphics accelerator, offering basic graphic display functionality.
- virtio-blk: VirtIO block device emulation, providing a virtual hard disk with an efficient I/O architecture for storing the file system and data of virtual machines.

These devices were chosen to assess and demonstrate QEMU's performance and compatibility across various

virtualization environments, furthering the development and application of virtualization technology.

Table II provides valuable statistics on line, function, and branch coverages. The hypervisor-specific solutions in DetecVFuzz drive fuzzing exploration towards the application execution stage, and the results demonstrate the contributions of DetecVFuzz to code coverage.

The findings indicate that our framework can be adapted to various types of devices, including USB, network, audio, storage, etc., which underscores the scalability of DetecVFuzz. It's crucial to emphasize that the entire process of implementing fuzzing for each device is entirely automated, requiring no human intervention. This remarkable feature is attributed to our DMA hooking solutions. By replacing the data interaction interfaces with fuzzing inputs, the process of fuzzing the hypervisor becomes akin to fuzzing an application, making it well-suited for coverage-guided fuzzers such as AFL, libfuzzer, and others.

Nonetheless, there are areas of code coverage that are not included in Table II, primarily due to two factors: (1) some devices were not tested at all; (2) certain code snippets can only be covered with specific emulated architectures (like Arm/PPC/MIPS/...) and specific startup configurations.

The last column in Table II indicates the number of executions per second. As expected, DetecVFuzz achieves an

impressive throughput of 6314.78 executions per second on average. This high throughput is due to our lightweight design, which eliminates the need for fork() or restarting the hypervisor. Furthermore, since we integrated the fuzzing agent into the hypervisor instead of running it in the guest OS, the fuzzer expends minimal time on data interaction. This approach is similar to fuzzing running on general applications and offers advantages over other fuzz designs that operate within a kernel module.

In the field of consumer electronics, such as smartphones, key hardware standards play a vital role. The related devices and standards in Table II not only determine how the device is connected to external devices, but also directly affect the performance and user experience of the device.

Smartphones usually adopt XHCI or other corresponding USB controller standards to achieve compatibility with various USB devices. This includes chargers, external storage devices, keyboards, and more. XHCI, as a modern USB standard, provides devices with higher data transfer rates and more advanced features, thereby improving the user experience when connecting external devices.

In addition, for the main storage interface, Android mobile devices usually choose NVME. It is a storage solution designed to provide faster read and write speeds and better performance. NVME is particularly suitable for high-performance needs, such as providing faster data transfer rates when processing large files, applications, and games.

Therefore, these hardware standards are directly related to consumer expectations for smartphone performance and functionality. Fuzz testing of modern USB controllers and high-performance storage interfaces can help improve the security and user experience of consumer electronics products. In summary, DetecVFuzz effectively enhances code coverage and scalability with limited additional costs.

B. Effectiveness of the Structure Automation Detection Module

To assess the specific effects of DetecVFuzz and its structure automation detection module, we conducted an evaluation that compared DetecVFuzz (without structure automation detection, directly starting fuzzing) to DetecVFuzz-p (with structure automation detection enabled). We also included a basic Qtest-based fuzzing approach as a baseline. This basic approach resembles earlier fuzzing methods that lack an understanding of the deep nesting characteristics of the target functions, and they attempt to randomly mutate the basic input interfaces (MMIO and DMA) without structural awareness.

In addition, we implemented a structure-aware fuzzing approach (referred to as Q2) that targets these devices through a generative fuzzing method. This structure-aware fuzzing was manually created based on the data structure specifications in the documentation. It involved building nested structures and establishing relationships between different types of nodes. The custom handmade fuzzing with structural awareness followed these steps: (1) Initialization of devices and registration of the initial status through MMIO and PIO. (2) Selection of a random structure and generation of random data to

populate it. Injection of the new data structure into the device, activating control commands for data execution. (3) As a point of comparison, we also assessed the throughput of a basic Qtest-based fuzzing approach (our baseline), which simply injects random data into basic interfaces (MMIO and DMA) without understanding the nested DMA structures.

To ensure a meaningful comparison, we selected target devices based on the following criteria:

(a) The target devices frequently utilize DMA operations, making them more suitable for evaluating the impact of the DetecVFuzz method since our focus is on DMA. Improved fuzzing performance can be observed as long as the device employs DMA.

(b) The safety and criticality of the devices are significant. These devices should be widely used, often included by default, and any vulnerability that affects the security of virtual machines should have already been discovered. We restricted our comparison to the following devices: PCNET, SDHCI, disk floppy, intel-hda, and USB EHCI.

For the evaluation, we ran DetecVFuzz (without structure detection), DetecVFuzz-p (with structure detection), handmade structure-aware fuzzing, and basic dumb Qtest on QEMU for 24 hours with 8 runs. The target hypervisor was executed with gcov, and gcov_flush() was called every second to record the coverage found over time. The results, presented in Fig. 7, display branch coverage in a log scale over the testing time.

As shown in Fig. 7, our DetecVFuzz-p can achieve the highest coverage and go through the deep path faster. The crude dumb Qtest performs the worst results, due to the fact that it loses every information of the input structure of the fuzzing targets, stopping its exploration at the very start of the program state. DetecVFuzz-p also discovers more branches than handmade fuzzing after the first hour. On EHCI, the handmade structure-aware fuzzing discovers about 65.8% of branches in 24 hours, while DetecVFuzz-p discovers 72.1%. On PCNET, the handmade structure-aware fuzzing discovers about 62.1% of branches in 24 hours, while DetecVFuzz-p discovers 83.3%. On SDHCI, the handmade structure-aware fuzzing discovers about 65.0% of branches in 24 hours, while DetecVFuzz-p discovers 86.9%. According to our analysis, we can give the credit to these factors: (1) DetecVFuzz's feature could cover deep branches caused by continual use of pci_dma_read function, where its argument address cannot be predicted, revealing an unknown type structure with the multi-layered feature. (2) Our structure detection fuzzing is sometimes unable to parse certain data structures, due to the diversities of the device specifications. As a result, we can confirm that, unlike the automatic and accurate nature of DetecVFuzz-p, writing manual rules by human effort is subject to error just like time-consuming tasks.

C. Fuzzing Against Other Methods

Next, we conducted a comparison between DetecVFuzz (with structure detection fuzzing mode enabled) and existing hypervisor fuzzers, including dumb Qtest, Nyx, Hyper-Cube, and VDF. VDF integrates the AFL workflow into VM fuzzing tests and is specifically designed for VM initialization.

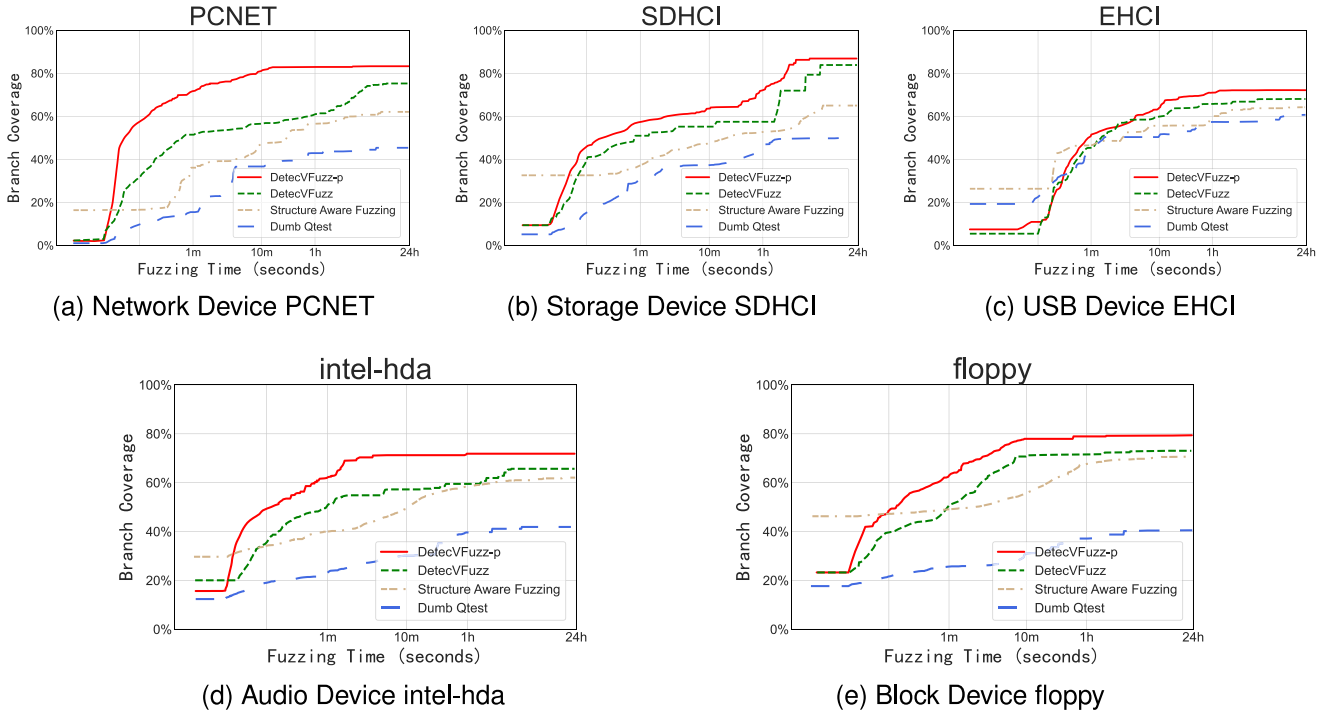


Fig. 7. Fuzzing with different DetecVFuzz modules combination and branch coverage results on QEMU device during 24 hours of test execution.

TABLE III
BRANCH COVERAGE DETECTED BY DETECVFUZZ COMPARED TO RESULTS OF VDF, HYPER-CUBE, NYX, HYPERCUBE ON VIRTUAL DEVICES

Device type	name	dumb Qtest	DetecVFuzz-1h	DetecVFuzz	DetecVFuzz-p	Nyx	VDF	Hyper-Cube
Block	Floppy	41.96	52.96	74.15	79.17	83.10	70.50	77.48
USB	UHCI	10.85	23.85	43.19	57.32	63.21	/	54.30
USB	XHCI	9.88	38.30	47.82	55.96	69.93	/	64.70
Storage	SDHCI	47.74	69.61	82.74	89.07	88.93	90.50	81.15
SCSI	Megass	28.32	36.22	40.21	54.38	/	/	45.28
Network	PCNET	38.80	55.93	78.48	89.22	89.49	36.10	78.71
Network	e1000	47.30	59.08	65.92	78.37	54.55	81.60	66.08
Audio	intel-hda	52.47	54.90	73.48	80.12	78.33	58.60	79.17

Nyx delves into the protocol and nested data structures for improved guidance. However, Nyx's results are only available from their original paper due to the lack of open-source code, so we could only compare it using the data published for certain devices. Hyper-Cube implements a custom operating system to interact with VM interfaces in any order. Additionally, we included DetecVFuzz-p (as mentioned earlier) and DetecVFuzz-1h (the complete version but running tests for only 1 hour) in the comparison.

The comprehensive results are presented in Table III. Our approach outperforms these devices compared to VDF, even though the targets differed after the update. In comparison to Nyx, we observed only marginal improvement. However, when the firmware and drivers adopt a sufficient number of DMA operations, DetecVFuzz can outperform Nyx in terms of coverage. This suggests that the fuzzing technique might have reached its limits in the case of small-scale firmware code, where most paths are superficial and easily triggered. When compared to Hyper-Cube, which incorporates a handcrafted lightweight operating system, we notice that this high-throughput but black-box approach encounters imperfections, whereas our method achieves greater coverage. As the quantity and complexity of the targets increase, we attribute

the superior performance to our DMA hooking approach, which unwraps nested structures and aids in the discovery of deeper code paths.

D. Vulnerability Hunting

To show the effectiveness of our method in the real world, we collect the bugs and crash information to demonstrate that our DetecVFuzz framework is capable of catching real vulnerabilities. To be more clear, the question is: can DetecVFuzz find new bugs and vulnerabilities in different virtual devices?

An overview of the types of crashes found is presented in Table IV. As demonstrated in the table, DetecVFuzz has successfully identified 15 new and unique crashes related to memory vulnerabilities. These bugs encompass nearly all common types of memory errors and are associated with various device types typically found in hypervisors. This illustrates how DetecVFuzz can enhance hypervisor security from multiple perspectives. Notably, many of the existing examples of virtual machine escapes result from buffer overflows and use-after-free bugs, making them potentially exploitable.

TABLE IV
TYPES STATISTICS OF UNIQUE CRASHES FOUND BY DETECVFUZZ

Type	#Bugs
Use After Free	1
Heap Buffer Overflow	3
Stack Overflow	1
Segmentation Fault	5
Out of Boundary	2
Infinite loop	3
#Total	15

Subsequently, we conducted an in-depth analysis of the program crashes and identified potential security issues within the virtual machine hardware. DetecVFuzz detected two vulnerabilities previously identified by researchers during actual testing: CVE-2021-3507, which is a heap buffer overflow in the floppy disk controller, and CVE-2020-25625, which involves an infinite loop in the OHCI (USB On-The-Go) driver. Although it is convenient to count the number of crash or hang test cases found, a comprehensive understanding of the virtual device under test is necessary. In addition, obtaining an effective Proof of Concept (PoC) is essential to trigger the corresponding vulnerabilities. It is important to note that many crashes only lead to program termination without posing actual security problems. Therefore, security researchers delve further to analyze those errors with possible implications for the exploitable attack surface.

The process of test case minimization significantly simplifies this analysis. Many vulnerabilities identified by DetecVFuzz are minimized to a common set of read and write operations. While the order of these operations may differ, the essential read/write operations that ultimately trigger the vulnerability remain consistent. The detection of documented vulnerabilities by DetecVFuzz (i.e., CVE-2021-3507 for floppy disk heap buffer overflow and CVE-2020-25625 for OHCI infinite loop) demonstrates its practicality in the industrial sector.

VI. DISCUSSION

Challenges of Complex Data Structures: We need to acknowledge that in our DetecVFuzz, we have good branch coverage compared to other fuzz tests, but it is a challenge to implement a fully automated fuzz loop in virtual hardware fuzz testing. There are still uncovered lines of code. This is a common problem with virtual hardware fuzzing today, and our automated testing process needs to be further improved to cover more complex data structures.

Capturing Errors During Virtual Machine Restart and Initialization: Furthermore, certain vulnerabilities are associated with errors that occur during the reset and startup of the virtual machine. To catch this part of the anomaly, we tried to re-enter the reset state every ten times during the test to initialize the reset. This captures errors during VM resets and startups without greatly reducing the efficiency of fuzz testing, and the specific results are basically the same as the original method.

Manually Creating Valid Proof of Concept (PoC): Our approach also seeks to eliminate the need for manually writing

rules during fuzzing. Yet, similar to the majority of fuzzing methods, it requires the expertise of professionals and human resources to confirm vulnerabilities and create valid Proof of Concept (PoC) exploits from the discovered crashes.

Challenges of Commercial Closed-Source Virtual Machines: Additionally, DetecVFuzz necessitates customizing the hypervisor source code to re-establish the connection between the device's data requests and the external fuzzer through MMIO/PIO and DMA interfaces. This customization is challenging for closed-source commercial virtual machines.

Impact of Device State Changes on Coverage Guidance: It's important to note that altering the internal states of the target device can introduce instability in the coverage guidance process. The same input may exercise different code paths depending on the hypervisor's state. Furthermore, when changes to persistent states accumulate, the device may eventually become unresponsive.

VII. CONCLUSION

Nowadays, consumer electronics is growing rapidly, and device virtualization has become critical in terms of efficiency and user experience. These virtual devices represent an attack surface through which software vulnerabilities [8] within the hypervisor can be exploited. This paper introduces DetecVFuzz, a scalable testing framework designed to fuzz virtual devices in QEMU. The approach hinges on comprehensive functions and input analysis to achieve this goal. DetecVFuzz not only automates the fuzzing of programs across various virtual devices but also does so by leveraging coverage-guided fuzzing. In addition to this, DetecVFuzz effectively performs lightweight multi-seed pool fuzzing, which helps mitigate the challenges associated with intricate input specifications across different devices and the allocation of additional computational resources. To evaluate the performance of DetecVFuzz, we applied it to the QEMU architecture along with its corresponding virtual devices. This process led to the discovery of 15 new memory-related bugs and errors. Notably, three of these have been validated as new vulnerabilities by the broader cybersecurity community.

REFERENCES

- [1] J. Shen, W. Yang, Z. Chu, J. Fan, D. Niyato, and K.-Y. Lam, "Effective intrusion detection in heterogeneous Internet-of-Things networks via ensemble knowledge distillation-based federated learning," 2024, *arXiv:2401.11968*.
- [2] M. Sayad Haghighi, F. Farivar, A. Jolfaei, A. B. Asl, and W. Zhou, "Cyber attacks via consumer electronics: Studying the threat of covert malware in smart and autonomous vehicles," *IEEE Trans. Consum. Electron.*, vol. 69, no. 4, pp. 825–832, Nov. 2023.
- [3] X. Hu et al., "Event detection in online social network: Methodologies, state-of-art, and evolution," *Comput. Sci. Rev.*, vol. 46, Nov. 2022, Art. no. 100500.
- [4] C. Abdoul-Kader and S.-H. Chang, "A novel venom attack identification mechanism in cloud virtualization environment," *Commun. CCISA*, vol. 24, no. 1, pp. 61–72, 2018.
- [5] J. Zhang, L. Pan, Q.-L. Han, C. Chen, S. Wen, and Y. Xiang, "Deep learning based attack detection for cyber-physical system cybersecurity: A survey," *IEEE/CAA J. Automatica Sinica*, vol. 9, no. 3, pp. 377–391, Mar. 2022.
- [6] N. Sun, J. Zhang, P. Rimba, S. Gao, L. Y. Zhang, and Y. Xiang, "Data-driven cybersecurity incident prediction: A survey," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 2, pp. 1744–1772, 2nd Quart., 2018.

- [7] J. Qiu, J. Zhang, W. Luo, L. Pan, S. Nepal, and Y. Xiang, "A survey of android malware detection with deep neural models," *ACM Comput. Surv.*, vol. 53, no. 6, pp. 1–36, 2020.
- [8] G. Lin, S. Wen, Q.-L. Han, J. Zhang, and Y. Xiang, "Software vulnerability detection using deep neural networks: A survey," *Proc. IEEE*, vol. 108, no. 10, pp. 1825–1848, Oct. 2020.
- [9] X. Chen et al., "Android HIV: A study of repackaging malware for evading machine-learning detection," *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 987–1001, 2020.
- [10] R. Dong, B. Cui, Y. Sun, and J. Yang, "BTIDEC: A novel detection scheme for cpu security of consumer electronics," *IEEE Trans. Consum. Electron.*, vol. 70, no. 1, pp. 4515–4523, Feb. 2024.
- [11] Y. Zhang et al., "Detecting union type confusion in component object model," in *Proc. 32nd USENIX Security Symp. (USENIX Security)*, 2023, pp. 4265–4281.
- [12] X. Zhu, S. Wen, A. Jolfaei, M. S. Haghighi, S. Camtepe, and Y. Xiang, "Vulnerability detection in IIoT applications: A fuzzing method on their binaries," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 3, pp. 970–979, May/Jun. 2022.
- [13] X. Zhu, S. Wen, S. Camtepe, and Y. Xiang, "Fuzzing: A survey for roadmap," *ACM Comput. Surv.*, vol. 54, no. 11, pp. 1–36, 2022.
- [14] A. Henderson, H. Yin, G. Jin, H. Han, and H. Deng, "VDF: Targeted evolutionary fuzz testing of virtual devices," in *Proc. Int. Symp. Res. Attacks, Intrusions, Def.*, 2017, pp. 3–25.
- [15] S. Schumilo, C. Aschermann, A. Abbasi, S. Wörner, and T. Holz, "HYPER-CUBE: High-dimensional hypervisor fuzzing," in *Proc. NDSS*, 2020, pp. 1–16.
- [16] S. Schumilo, C. Aschermann, A. Abbasi, S. Wörner, and T. Holz, "NYX: Greybox hypervisor fuzzing using fast snapshots and affine types," in *Proc. USENIX Security Symp.*, 2021, pp. 2597–2614.
- [17] B. P. Miller, L. Fredriksen, and B. So, "An empirical study of the reliability of UNIX utilities," *Commun. ACM*, vol. 33, no. 12, pp. 32–44, 1990.
- [18] S. Rawat, V. Jain, A. Kumar, L. Cojocar, C. Giuffrida, and H. Bos, "VUZZer: Application-aware evolutionary fuzzing," in *Proc. NDSS*, vol. 17, 2017, pp. 1–14.
- [19] C. Aschermann, T. Frassetto, T. Holz, P. Jauernig, A.-R. Sadeghi, and D. Teuchert, "NAUTILUS: Fishing for deep bugs with grammars," in *Proc. NDSS*, 2019, pp. 1–15.
- [20] C. Aschermann, S. Schumilo, T. Blazytko, R. Gawlik, and T. Holz, "REDQUEEN: Fuzzing with input-to-state correspondence," in *Proc. NDSS*, vol. 19, 2019, pp. 1–15.
- [21] Y. Chen et al., "EnFuzz: Ensemble fuzzing with seed synchronization among diverse fuzzers," in *Proc. USENIX Security Symp.*, 2019, pp. 1967–1983.
- [22] A. Rebert et al., "Optimizing seed selection for fuzzing," in *Proc. 23rd USENIX Security Symp.*, 2014, pp. 861–875.
- [23] I. Yun, S. Lee, M. Xu, Y. Jang, and T. Kim, "QSYM: A practical concolic execution engine tailored for hybrid fuzzing," in *Proc. 27th USENIX Security Symp.*, 2018, pp. 745–761.
- [24] D. Song, F. Hetzelt, J. Kim, B. B. Kang, J.-P. Seifert, and M. Franz, "Agamotto: Accelerating kernel driver fuzzing with lightweight virtual machine checkpoints," in *Proc. 29th USENIX Conf. Security Symp.*, 2020, pp. 2541–2557.
- [25] D. Song et al., "Periscope: An effective probing and fuzzing framework for the hardware-os boundary," in *Proc. Netw. Distrib. Syst. Security Symp. (NDSS)*, 2019, pp. 1–15.
- [26] H. Peng and M. Payer, "USBfuzz: A framework for fuzzing USB drivers by device emulation," in *Proc. 29th USENIX Conf. Security Symp.*, 2020, pp. 2559–2575.
- [27] A. Fioraldi, D. Maier, H. Eißfeldt, and M. Heuse, "AFL++ combining incremental steps of fuzzing research," in *Proc. 14th USENIX Conf. Offensive Technol.*, 2020, p. 10.
- [28] C. Lattner, "LLVM and clang: Next generation compiler technology," in *Proc. BSD Conf.*, vol. 5, 2008, pp. 1–20.
- [29] K. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov, "AddressSanitizer: A fast address sanity checker," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC)*, 2012, pp. 309–318.
- [30] D. Li and H. Chen, "FastSyzykaller: Improving fuzz efficiency for linux kernel fuzzing," *J. Phys., Conf. Series*, vol. 1176, no. 2, 2019, Art. no. 22013.
- [31] P. Pandey, A. Sarkar, and A. Banerjee, "Triforce QNX syscall fuzzer," in *Proc. IEEE Int. Symp. Softw. Reliab. Eng. Workshops (ISSREW)*, 2019, pp. 59–60.
- [32] B. Garn and D. E. Simos, "Eris: A tool for combinatorial testing of the linux system call interface," in *Proc. IEEE 7th Int. Conf. Softw. Test., Verif. Valid. Workshops*, 2014, pp. 58–67.
- [33] J. Corina et al., "Difuze: Interface aware fuzzing for kernel drivers," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2017, pp. 2123–2138.
- [34] S. Schumilo, C. Aschermann, R. Gawlik, S. Schinzel, and T. Holz, "kAFL: Hardware-Assisted feedback fuzzing for OS kernels," in *Proc. 26th USENIX Security Symp.*, Aug. 2017, pp. 167–182. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/schumilo>
- [35] D. R. Jeong, K. Kim, B. Shivakumar, B. Lee, and I. Shin, "Razzer: Finding kernel race bugs through fuzzing," in *Proc. IEEE Symp. Security Privacy (SP)*, 2019, pp. 754–768.
- [36] H. Han and S. K. Cha, "IMF: Inferred model-based fuzzer," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2017, pp. 2345–2358.
- [37] K. Kim, D. R. Jeong, C. H. Kim, Y. Jang, I. Shin, and B. Lee, "HFL: Hybrid fuzzing on the linux kernel," in *Proc. NDSS*, 2020, pp. 1–17.
- [38] M. Xu, S. Kashyap, H. Zhao, and T. Kim, "Krace: Data race fuzzing for kernel file systems," in *Proc. IEEE Symp. Security Privacy (SP)*, 2020, pp. 1643–1660.
- [39] W. Xu, H. Moon, S. Kashyap, P.-N. Tseng, and T. Kim, "Fuzzing file systems via two-dimensional input space exploration," in *Proc. IEEE Symp. Security Privacy (SP)*, 2019, pp. 818–834.
- [40] J. Chen et al., "IoTfuzzer: Discovering memory corruptions in IoT through app-based fuzzing," in *Proc. NDSS*, 2018, pp. 1–15.
- [41] A. Clements et al., "HALucinator: Firmware re-hosting through abstraction layer emulation," in *Proc. 29th USENIX Security Symp.*, 2020, pp. 1–18.
- [42] X. Feng, X. Zhu, Q.-L. Han, W. Zhou, S. Wen, and Y. Xiang, "Detecting vulnerability on IIoT device firmware: A survey," *IEEE/CAA J. Automatica Sinica*, vol. 10, no. 1, pp. 25–41, Jan. 2023.
- [43] R. Van Tonder and H. A. Engelbrecht, "Lowering the USB fuzzing barrier by transparent two-way emulation," in *Proc. WOOT*, 2014, pp. 1–8.
- [44] S. Schumilo, C. Aschermann, A. Jemmett, A. Abbasi, and T. Holz, "Nyx-Net: Network fuzzing with incremental snapshots," in *Proc. 17th Eur. Conf. Comput. Syst.*, 2022, pp. 166–180.
- [45] T. Rosado and J. Bernardino, "An overview of openstack architecture," in *Proc. 18th Int. Database Eng. Appl. Symp.*, 2014, pp. 366–367.
- [46] G. Klees, A. Ruef, B. Cooper, S. Wei, and M. Hicks, "Evaluating fuzz testing," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2018, pp. 2123–2138.
- [47] H. Green and T. Avgerinos, "GraphFuzz: Library API fuzzing with lifetime-aware dataflow graphs," in *Proc. 44th Int. Conf. Softw. Eng.*, 2022, pp. 1070–1081.